

Enhancing Cross-Project Test Smell Detection via Test Code Feature Engineering and Ensemble Learning

Nguyen Thi Thuy Hoai¹[0009-0007-6328-0715], Nguyen Nang Hung Van²[0000-0002-9963-7006], and Nguyen Thanh Binh³[0000-0002-0154-1162] *

¹ FPT University, Danang, Vietnam
hoaintt40@fe.edu.vn

² The University of Danang - University of Science and Technology, Danang, Vietnam
nguyenvan@dut.udn.vn

³ The University of Danang - Vietnam-Korea University of Information and Communication Technology, Danang, Vietnam
ntbinh@vku.udn.vn

Abstract. In software development, testing is a crucial process that ensures the quality and maintainability of software systems. Recent research shows that test smells in test cases negatively affect the cost and overall health of software projects. Earlier research on test smell detection has largely relied on heuristic or rule-based techniques, which depend on subjectively defined thresholds and often fail to generalize across projects. More recent studies have introduced machine learning (ML) to automate detection; however, their effectiveness remains limited because they rely almost exclusively on features extracted from test code. To address these limitations, this study makes two key contributions. First, we construct an enriched dataset by extending an existing ground-truth corpus of 9,633 manually labeled Java test cases, linking each test class. Second, we propose an enhanced detection framework based on ensemble learning, combined with test code feature selection, and evaluate it across 59 Java projects covering four test smells—Eager Test, Mystery Guest, Resource Optimism, and Test Redundancy. Our results demonstrate substantial improvements in F1-score compared to previously reported cross-project studies, indicating significantly stronger generalization and more robust detection performance.

Keywords: test smell · test smell detection · machine learning · stacking ensemble technique.

1 Introduction

Testing, particularly automated unit testing, plays a crucial role in the software development life cycle [19]. It ensures software quality by evaluating the system under test (SUT) through controlled execution to reveal failures, helping

* Corresponding author, Email: ntbinh@vku.udn.vn

verify that software artifacts behave as intended and do not exhibit unintended actions [16]. Unit testing also supports regression prevention and serves as a cornerstone of agile methodologies such as Test-Driven Development and eXtreme Programming, where continuous verification is essential to avoid behavioral degradation during evolution [3, 4].

Similar to production code, test code can also suffer from quality issues known as test smells [1]. These smells are symptoms of poorly designed test cases or suboptimal testing practices and are known to negatively affect maintainability, reliability, and long-term test suite evolution [19, 27]. Detecting test smells is therefore an essential activity for improving the effectiveness of automated testing and supporting CI/CD workflows.

Traditional detection approaches are largely heuristic and rely on manually defined thresholds over metrics extracted exclusively from test code [23]. While such rule-based methods are simple to implement, they suffer from limited generalizability because (i) thresholds often do not transfer well across projects, and (ii) heuristic rules may fail to capture complex structural patterns underlying test smells.

To overcome these limitations, recent works have applied machine learning (ML) to detect test smells using features derived solely from test cases. Prior studies have demonstrated the feasibility of ML-based detection for smells including Eager Test, Mystery Guest, Resource Optimism, and Test Redundancy [23]. Other research has explored deep learning, autoencoders, and multi-label classification for automatically identifying test-code quality issues [21]. Ensemble-learning methods such as stacking have shown further accuracy improvements compared with single-model approaches [35]. However, existing ML-based works still rely exclusively on test-code metrics, and challenges remain regarding cross-project generalization and performance on highly imbalanced smell categories.

To address these challenges, this study focuses on cross-project test smell detection using carefully selected test-code features and ensemble learning techniques. In particular, we aim to systematically analyze the effectiveness of minimal and enriched test-code feature sets under a Leave-One-Project-Out (LOPO) evaluation strategy. The study is guided by the following research questions:

RQ1. How effective are minimal test-code features in detecting test smells under a cross-project setting? This question investigates whether a small set of interpretable test-code features can achieve competitive performance compared to existing baselines when applied to unseen projects.

RQ2. To what extent do additional test-code OO metrics improve cross-project test smell detection performance? This question examines whether incorporating structural and complexity-related metrics from test code enhances robustness and generalization across heterogeneous projects.

RQ3. Can an ensemble learning strategy mitigate class imbalance and improve detection performance across different types of test smells? This question explores the capability of stacking-based classifiers to deliver stable performance, particularly for minority test smells in highly imbalanced datasets.

By answering these research questions, this work aims to provide empirical insights into the design of effective and generalizable ML-based test smell detectors that rely solely on test-code characteristics.

2 Background

2.1 Metrics in test code

Metrics play a crucial role in characterizing test code often serving as the foundation for automated detection and quantitative analysis of test quality, particularly concerning test smells [2, 5]. Traditionally, maintainability is assessed using internal code metrics relating to size, complexity, coupling, and cohesion [2]. Researchers have proposed and utilized specific metrics, often defined through a formal meta-model based on the xUnit family of frameworks, to quantify aspects of test design flaws. There are two types of metrics code for detecting specific test smell: structural metrics and object-oriented metrics.

Structural metrics These metrics-based heuristic approaches evaluate specific properties of unit tests against empirically derived thresholds to determine the presence and relative severity of test smells. The metrics fall into four categories: Fixture Metrics [2], Focus Metrics [6, 18], Readability Metrics [5] and Cohesion Metrics [15].

Object-oriented metrics The quantitative evaluation of test code relies on metrics adapted from object-oriented (OO) analysis or defined specifically for test artifacts. While traditional OO metrics (e.g., CC, WMC) are often applied to production code, several specialized metrics are computed directly on test classes, based on a meta-model distinguishing test code, production code, and external libraries in xUnit environments. Lack of Cohesion of Test Methods (LCOTM) adapts OO cohesion metrics to assess the internal structure of a test class. It measures cohesion only among test methods and considers all fields accessed by them, including ad hoc fields created within tests. Values range from 0 (cohesive) to 1 (incohesive); $LCOTM \geq 0.4$ indicates a smelly class. indicates a smelly class.

Fixture metrics capture structural properties of the test fixture and support detection of the General Fixture smell (NFOB, NFPT), Complexity and Size Metrics quantify complexity within test methods (Decision Points, SLOC) and Obscure In-line Setup based on the number of local variables in a test method; large counts (e.g., ≥ 10) suggest hidden setup logic.

2.2 Formulas of OO metrics

To ensure clarity and reproducibility, we present formal metric formulas which will be employed in this study [31].

Notation Let a class C provide a method set $M = \{m_1, \dots, m_n\}$ and attribute set $I = \{a_1, \dots, a_k\}$. Each method m_i accesses a subset $I_i \subseteq I$; $Calls(m_i)$ denotes methods invoked by m_i .

1) WMC – Weighted Methods per Class

$$WMC(C) = \sum_{i=1}^n c(m_i)$$

where $c(m_i)$ denotes method complexity (commonly cyclomatic complexity). Higher WMC implies higher implementation and testing effort.

2) DIT – Depth of Inheritance Tree

$$DIT(C) = \max(\text{length of inheritance path from } C \text{ to root})$$

A large DIT indicates deeper inheritance and increased behavioral complexity.

3) LOC – Lines of Code

$$LOC(C) = N_{\text{lines}}^{\text{test}} - N_{\text{blank}}^{\text{test}} - N_{\text{comment}}^{\text{test}}$$

LOC denotes the number of effective lines of executable code in a test class, excluding blank and comment lines..

4) CBO – Coupling Between Objects

$$CBO(C) = \left| \{ X \neq C \mid C \leftrightarrow X \} \right|$$

where $C \leftrightarrow X$ indicates any usage relationship (calls, field access, or type references).

5) RFC – Response For a Class

Let the response set be:

$$RS(C) = M \cup \bigcup_{m \in M} Calls(m)$$

Then:

$$RFC(C) = |RS(C)|$$

Large RFC means more methods may execute in response to a message, implying higher testing complexity.

3 Related Work

Research on test smells has focused on defining smell catalogs, collecting empirical evidence, and developing automated detection methods. Early studies formalized core smells such as Eager Test and Mystery Guest using xUnit-based testing models [1,4]. Empirical analyses showed that smells are widespread—86% of JUnit tests contain at least one—and negatively impact maintainability [2,26], motivating the creation of validated smell datasets. Automated detection evolved from metric-based heuristics [3] to advanced IR and ML techniques [15], and more recently to LLM/SLM-based approaches that better capture code semantics [16,27–29].

3.1 Heuristic and Metrics-based approaches

Heuristic and metrics-based approaches utilize static analysis to quantify specific properties of test code and apply predefined rules and thresholds to identify structural deviations, or smells [2,3]. The goal is to clarify the structural deficiencies encapsulated in test smells by formalizing core test concepts and their characteristics. This involves specifying a meta-model for unit testing, typically based on concepts from the xUnit family of testing frameworks. Researchers formulate heuristics that measure unit test properties and rank tests based on the presence and significance of smells. These rules often consist of logically combined sets filtered on the elements’ metric values. These methods rely on quantifiable metrics derived from static code analysis, such as the Number of Production Type Uses (NPTU), the Number of Object Uses in Setup (NOBU), the number of assert statements, and other structural metrics related to cohesion, complexity, and coupling. For instance, metrics like $NPTU(m)$ quantify the number of production types invoked or referenced by a test method m or its helpers.

Besides, these metrics are used to formalize and detect specific smells like General Fixture and Eager Test. For example, detecting For Testers Only involves checking production methods invoked by test code but not by other production methods [2]. Detection tools based on heuristics, such as TestLint [2] and TestQ [27], require users to configure metrics thresholds to indicate the value from which a test entity exhibits a smell. The performance of these detectors is often strongly influenced by the thresholds used. However, early studies observed that metrics-based heuristics had limitations, yielding weak correlations with human reviewers’ ratings of smelliness. Manual inspection was necessary to check if fragments identified by exceptional metric values were truly smells.

3.2 ML-based approaches

Classic machine learning involves training standard classifiers on engineered features (metrics) to predict the likelihood of a test smell, avoiding the fixed thresholds of heuristic methods [21,23]. ML classifiers are trained using features derived from test code, including structural and textual metrics. Commonly investigated

classifiers (basic classifiers) include Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), Naive Bayes (NB) and Multilayer Perceptron (MLP). ML approaches have targeted smells like Eager Test, Mystery Guest, Resource Optimism and Test Redundancy. Despite being significantly better than heuristic methods, initial ML efforts based on engineered metrics achieved low performance, with an average F-Measure not exceeding 51% [23]. Ensemble methods like Random Forest and AdaBoost often show slightly higher median performance [21, 23]. The study test smell prediction with stacking ensemble learning reported an accuracy of approximately 0.87 and an F1-score of around 0.74 for its best-performing smell category, demonstrating clear improvements over conventional single-model baselines [35].

4 Proposed Approach

4.1 Dataset

In this study, we adopted and enriched an existing benchmark dataset consisting of 9,633 Java test cases that were manually validated for the presence of four types of test smells — Eager Test (ET), Mystery Guest (MG), Resource Optimism (RO), and Test Redundancy (TR) — as proposed in [23]. The labeling process was conducted by software testing experts, achieving a Cohen’s K coefficient of 0.84, indicating near-perfect inter-rater agreement and high reliability of the ground truth annotations.

4.2 Data Preprocessing and Cross-Project Evaluation

To extend and improve the dataset, we cloned 59 out of 66 original projects available in the repository. After preprocessing and cleaning invalid or missing test files, a total of 7,992 test cases were successfully reconstructed. The distribution of labeled instances across the four smell types. The distribution of test smell instances across the dataset is illustrated in Figure 1. The remaining test cases without smell annotation were considered as negative samples.

We evaluate cross-project generalization using a Leave-One-Project-Out (LOPO) strategy, where one project is held out as the test set and all remaining projects form the training set. For each smell label, only projects containing a sufficient number of positive instances are eligible as test sets, while extremely sparse projects are excluded to prevent unstable F1-score estimates.

All numerical features in Table 1 are standardized using StandardScaler to mitigate scale disparities across heterogeneous metrics. This label-aware LOPO protocol provides a reliable and realistic assessment of cross-project smell detection performance.

4.3 Model Architecture

The experimental process was structured into three sequential phases: (1) deep feature engineering and dataset construction, (2) data preprocessing and (3)

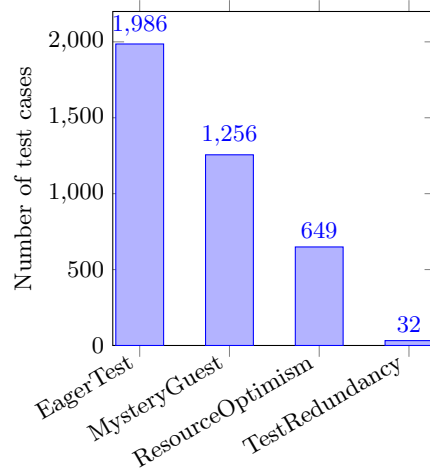


Fig. 1: The distribution of test smell instances across the dataset

stacking ensemble model configuration. This workflow for training and testing is described in Figure 2.

We employ a stacking ensemble classifier explicitly optimized for the minority smelly class, where the target objective is to maximize the smelly class F1-score. To reduce the impact of heavy-tailed feature distributions (e.g., LOC-related metrics) and project-level scale differences, we first standardize numeric features (and optionally apply robust scaling to mitigate outliers), ensuring that high-magnitude metrics do not dominate learning. Given the severe class imbalance observed across projects, the learning procedure is further biased toward smelly class via cost-sensitive weighting rather than naive accuracy optimization.

The base layer consists of three complementary learners: (i) a Random Forest with balanced subsampling to improve robustness under skewed label distributions, (ii) an XGBoost classifier configured with regularization and a tuned positive-class weight (`scale_pos_weight`) to favor recall for smelly class while controlling false positives, and (iii) a linear SVM implemented with SGD using a robust loss (`modified_huber`) and class-dependent weights to increase sensitivity to smelly instances. These models provide diverse inductive biases—tree ensembles capture non-linear interactions, boosting emphasizes hard examples, and the linear model offers stability in high-dimensional settings.

Their probabilistic outputs are combined by a Logistic Regression meta-classifier, trained under stronger class-1 weighting to further prioritize the detection of smelly test cases. To prevent information leakage and ensure stable meta-learning, we use 20-fold stratified stacking, where out-of-fold probabilities are generated for the training set and then used to fit the meta-learner. During inference, the final decision is produced from the stacked probability of smelly class, and the operating threshold is selected to maximize the smelly class F1-score (i.e., balancing precision and recall with an explicit emphasis on

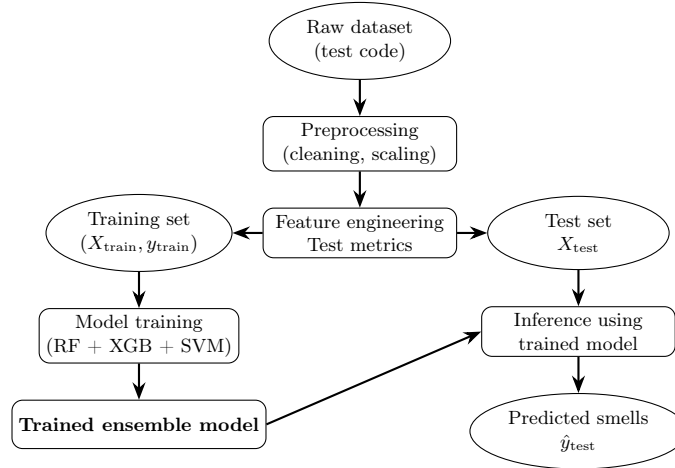


Fig. 2: Workflow of the proposed ML pipeline with a compact train–test separation

recall). This cost-sensitive, threshold-optimized stacking strategy consistently yields more reliable cross-project performance than individual classifiers, particularly for minority smells with high imbalance and heterogeneous project characteristics.

5 Experimental Results and Discussions

From the proposed approach above, we conducted two groups of experiments to systematically evaluate the effectiveness of our model. All experiments were performed on 59 Java projects using a Leave-One-Project-Out (LOPO) strategy to assess cross-project generalization.

Table 1 presents the test-code features employed in our experiments, comprising object-oriented metrics and smell-specific indicators derived from the study in [23]. Before analyzing the classification performance, we first examine the relationships among test-code features using correlation heatmaps (Figures 3 - 6). These results indicate that test-code structural features such as `test_rfc`, `test_wmc`, and `test_loc` are strongly correlated across all four test smells, suggesting that complex test cases tend to share similar structural characteristics. In contrast, smell-specific features (e.g., `SimilaritiesCoefficient` for Eager Test, `NRDB` for Mystery Guest, `ERNR` for Resource Optimism, and `PR` for Test Redundancy) exhibit weak correlations with structural metrics. This low correlation implies that these features capture complementary behavioral or contextual information rather than test complexity alone. Consequently, combining structural metrics with smell-specific features improves the model’s ability to generalize across projects and distinguish different types of test smells.

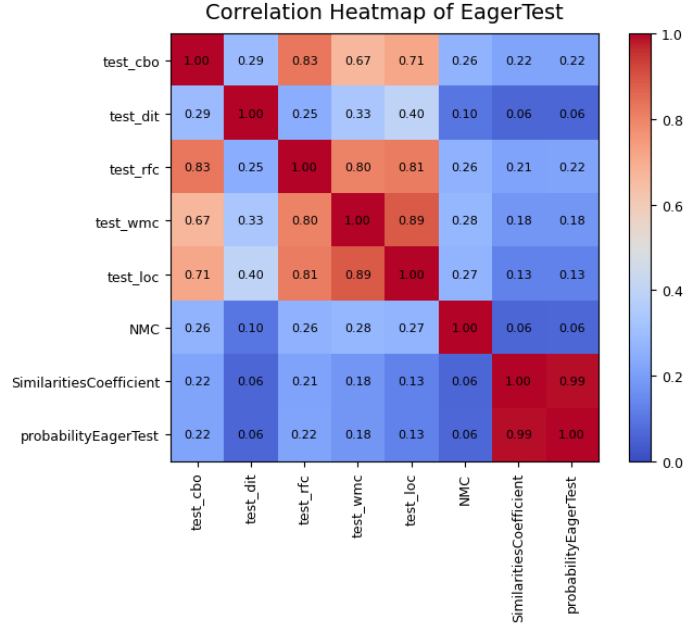


Fig. 3: Correlation heatmap for Eager Test (ET).

In the first step, we noticed the baseline model proposed in [23] using the original dataset and feature set. The reproduced results, reported in Table 2, are consistent with those presented in the original study, achieving F1-scores of 0.39 for Eager Test, 0.44 for Mystery Guest, 0.26 for Resource Optimism, and 0.01 for Test Redundancy.

In the second step, we applied our proposed learning framework while retaining only the minimal feature set defined in [23]. As shown in Table 2, this setting led to consistent performance improvements across all test smells. Specifically, the F1-score increased from 0.39 to 0.43 for Eager Test, remained 0.44 for Mystery Guest, from 0.26 to 0.48 for Resource Optimism, and from 0.01 to 0.11 for Test Redundancy, demonstrating the benefit of the proposed training strategy even without introducing additional features.

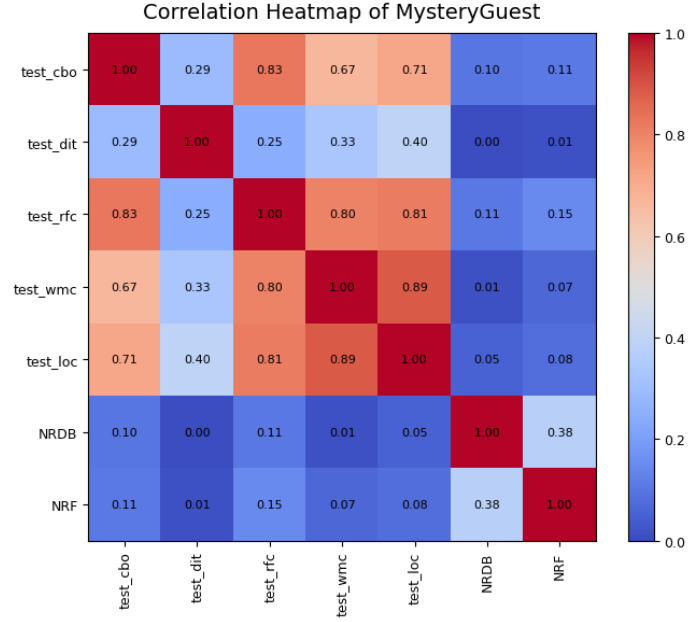


Fig. 4: Correlation heatmap for Mystery Guest (MG).

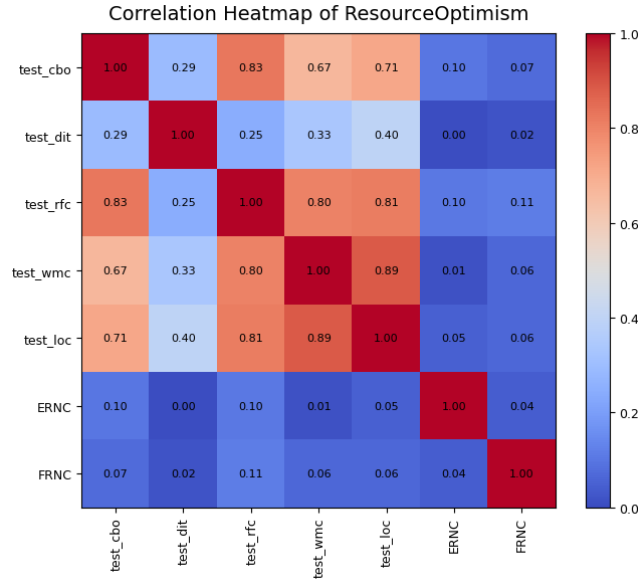


Fig. 5: Correlation heatmap for Resource Optimism (RO).

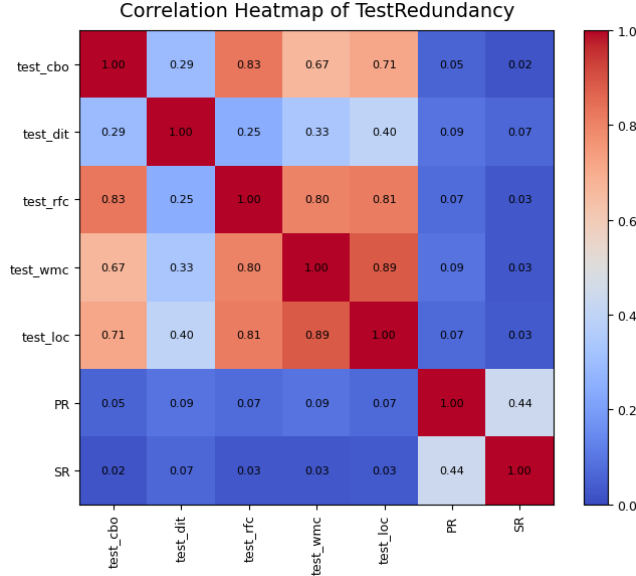


Fig. 6: Correlation heatmap for Test Redundancy (TR).

Finally, we further extended the minimal-feature models by incorporating five additional OO metrics extracted from test code (Table 1). The results, illustrated in Table 2 and Figures 7, demonstrate stronger cross-project performance for most test smells, achieving F1-scores of 0.4948 for Eager Test, 0.4674 for Mystery Guest, and 0.4306 for Resource Optimism. These improvements confirm that test-code structural complexity provides additional predictive value for test smell detection. Test Redundancy remained the most challenging smell with F1-score of 0.1107, largely due to extreme class imbalance and low separability between smelly and non-smelly instances, a trend also observed in prior studies.

Overall, the results demonstrate that model based on test code metrics outperform the baseline heuristic and ML models, and provide meaningful improvements in cross-project robustness.

6 Conclusion and Research Directions

In this study, we introduced a stacking ensemble learning-based test smell detector enriched with test code metrics. Through a comprehensive LOPO cross-project evaluation on four smell types, our model consistently outperformed the recent ML baseline, achieving notable F1-score improvements, especially for Eager Test, Resource Optimism, and Test Redundancy.

The experimental results also reveal that different test smells exhibit distinct feature sensitivities. While Eager Test, Mystery Guest, and Resource Optimism benefit from test-code complexity and interaction metrics, Test Redundancy

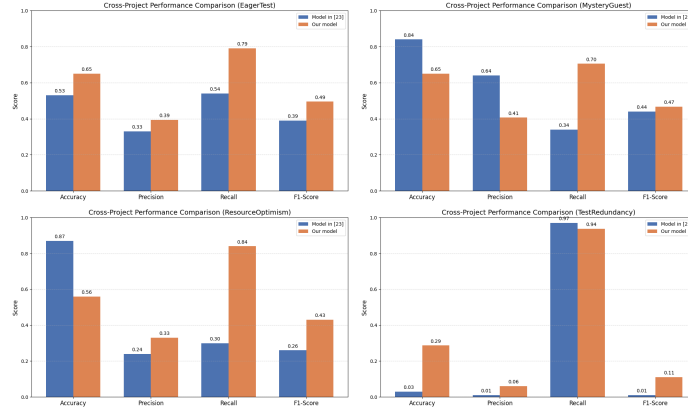


Fig. 7: Comparison between the proposed model and the baseline [23] for detecting test smells

remains difficult to detect due to extreme class imbalance and limited discriminative signals in test code alone. These findings emphasize the necessity of smell-specific modeling strategies and careful feature selection when addressing cross-project generalization.

Overall, the minimal feature and test code metrics model does not uniformly outperform the minimal feature model across all smells, but it provides the most balanced cross-project performance for Eager Test and Mystery Guest. While Resource Optimism slightly degrades precision due to increased false positives, indicating mild over-detection rather than severe overfitting. Importantly, no significant overfitting is observed under the LOPO setting, confirming that test-code metrics generally enhance generalization rather than memorization.

As future work, we intend to improve detection of rare smells with adding production code metrics and extending the proposed framework to multi-label detection.

Acknowledgment

This research is funded by the Vietnam National Foundation for Science and Technology Development (NAFOSTED) under grant number 102.03-2025.109.

Table 1: List of features used in our model, grouped by metric type and associated test smell.

Group	Test Smell	Feature	Description
OO Metrics(Test Code) All		test_cbo	Coupling between objects in the test class.
		test_dit	Depth of inheritance of the test class.
		test_rfc	Number of methods reachable from the test class.
		test_wmc	Weighted method complexity of the test class.
		test_loc	Lines of code of the test class or test method.
Metrics from [23]	Eager Test	NMC	Number of production methods called by the test.
		SimilaritiesCoefficient	Structural similarity between test cases.
		probabilityEagerTest	Prior probability of Eager Test derived from a baseline classifier.
Metrics from [23]	Mystery Guest	NRDB	Number of external or database resource dependencies.
		NRF	Number of potential resource-related failures.
Metrics from [23]	Resource Optimism	ERNC	Expected number of resource network calls.
		FRNC	Number of failed or risky resource network calls.
Metrics from [23]	Test Redundancy	PR	Positive resource usage indicating redundant execution.
		SR	Shared resource usage across multiple test cases.

Table 2: Comparison between the baseline model [23] and our proposed model with minimal and enriched test-code features across four test smells

Test Smell	Baseline [23]				Minimal-Feature				Minimal + Test-code Metrics				F1
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	
Eager Test	0.53	0.33	0.54	0.39	0.63	0.36	0.74	0.43	0.65	0.39	0.79		0.49
Mystery Guest	0.84	0.64	0.34	0.44	0.55	0.40	0.74	0.44	0.65	0.41	0.71		0.47
Resource Optimism	0.87	0.24	0.30	0.26	0.68	0.50	0.66	0.48	0.56	0.33	0.84		0.43
Test Redundancy	0.03	0.01	0.97	0.01	0.13	0.06	1.00	0.11	0.29	0.06	0.94		0.11

References

1. A. van Deursen and L. Moonen, “Refactoring test code,” in *Proc. Int. Conf. on Extreme Programming and Flexible Processes in Software Engineering (XP 2001)*, Aug. 2001, pp. 92–95. [Online]. Available: <https://homepages.cwi.nl/~arie/papers/xp2001.pdf>
2. B. V. Rompaey, B. Du Bois, and S. Demeyer, “Characterizing the relative significance of a test smell,” in *Proc. 22nd IEEE Int. Conf. on Software Maintenance (ICSM)*, Philadelphia, PA, USA, Sep. 2006, pp. 391–400, doi: 10.1109/ICSM.2006.18.
3. B. Van Rompaey, B. Du Bois, S. Demeyer, and M. Rieger, “On the detection of test smells: A metrics-based approach for general fixture and eager test,” *IEEE Trans. Softw. Eng.*, vol. 33, no. 12, pp. 800–817, Dec. 2007, doi: 10.1109/TSE.2007.70745.
4. G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Boston, MA, USA: Addison-Wesley, 2007. [Online]. Available: <https://martinfowler.com/books/meszaros.html>
5. M. Breugelmans and B. Van Rompaey, “TestQ: Exploring structural and maintenance characteristics of unit test suites,” in *Proc. 1st Int. Workshop on Academic Software Development Tools and Techniques (WASDeTT-1)*, Paphos, Cyprus, Jul. 2008. [Online]. Available: <https://scg.unibe.ch/download/wasdett/wasdett2008-paper07.pdf>
6. G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, “An empirical analysis of the distribution of unit test smells and their impact on software maintenance,” in *Proc. 28th IEEE Int. Conf. on Software Maintenance (ICSM)*, Riva del Garda, Italy, 2012, pp. 56–65, doi: 10.1109/ICSM.2012.6405253.
7. M. Greiler, A. van Deursen, and M.-A. D. Storey, “Automated detection of test fixture strategies and smells,” in *Proc. IEEE 6th Int. Conf. on Software Testing, Verification and Validation (ICST)*, Luxembourg, 2013, pp. 322–331, doi: 10.1109/ICST.2013.45.
8. M. Kummer, “Categorising test smells,” Bachelor’s thesis, Univ. of Bern, Software Composition Group, Bern, Switzerland, Mar. 2015. [Online]. Available: <https://scg.unibe.ch/archive/projects/Kumm15a.pdf>
9. M. Tufano, F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and D. Poshyanyk, “An empirical investigation into the nature of test smells,” in *Proc. 31st IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2016, pp. 4–15.
10. K. van den Bekerom, “Detecting test bugs using static analysis tools,” Master’s thesis, Univ. of Amsterdam, Amsterdam, The Netherlands, 2016.
11. F. Palomba, D. Di Nucci, A. Panichella, R. Oliveto, and A. De Lucia, “On the diffusion of test smells in automatically generated test code: An empirical study,” in *Proc. 9th Int. Workshop on Search-Based Software Testing (SBST ’16)*, Austin, TX, USA, 2016, pp. 5–14, doi: 10.1145/2897010.2897016.
12. N. Paul and R. Tommy, “An approach of automated testing on web based platform using machine learning and Selenium,” in *Proc. 2018 Int. Conf. on Inventive Research in Computing Applications (ICIRCA)*, Coimbatore, India, 2018, pp. 851–856, doi: 10.1109/ICIRCA.2018.8597297.
13. V. Garousi and B. Küçük, “Smells in software test code: A survey of knowledge in industry and academia,” *J. Syst. Softw.*, vol. 138, pp. 52–81, 2018, doi: 10.1016/j.jss.2017.12.013.

14. T. Sharma and D. Spinellis, “A survey on software smells,” *J. Syst. Softw.*, vol. 138, pp. 158–173, 2018.
15. J. Pantiuchina, G. Bavota, R. Oliveto, M. Di Penta, and B. Russo, “Automatic test smell detection using information retrieval techniques,” in *Proc. 2018 IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2018, pp. 311–322, doi: 10.1109/ICSME.2018.00038.
16. V. H. S. Durelli, R. S. Durelli, S. S. Borges, A. T. Endo, M. M. Eler, D. R. C. Dias, and M. P. Guimarães, “Machine learning applied to software testing: A systematic mapping study,” *IEEE Trans. Rel.*, vol. 68, no. 3, pp. 1189–1212, 2019, doi: 10.1109/TR.2019.2892517.
17. V. Garousi, B. Küçük, and M. Felderer, “What we know about smells in software test code,” *IEEE Softw.*, vol. 36, no. 3, pp. 61–73, 2019, doi: 10.1109/MS.2018.2875843.
18. D. J. Kim, “An empirical study on the evolution of test smell,” in *Proc. ACM/IEEE 42nd Int. Conf. on Software Engineering: Companion Proc. (ICSE-Companion)*, 2020, pp. 149–151.
19. N. Silva Junior, L. Rocha, L. A. Martins, and I. Machado, “A survey on test practitioners’ awareness of test smells,” *arXiv preprint arXiv:2003.05613*, 2020. Available: <https://arxiv.org/abs/2003.05613>
20. A. Tahir, S. Counsell, and S. G. MacDonell, “An empirical study into the relationship between class features and test smells,” in *Proc. 23rd Asia-Pacific Software Engineering Conference (APSEC)*, Hamilton, New Zealand, 2016, pp. 137–144, doi: 10.1109/APSEC.2016.34.
21. M. Hadj-Kacem and N. Bouassida, “A multi-label classification approach for detecting test smells over Java projects,” *J. King Saud Univ. — Comput. Inf. Sci.*, vol. 34, no. 10, pt. A, pp. 8692–8701, 2022, doi: 10.1016/j.jksuci.2021.10.008.
22. L. A. Martins, H. A. X. Costa, and I. d. C. Machado, “On the diffusion of test smells and their relationship with test code quality of Java projects,” *J. Softw.: Evol. Process*, vol. 36, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:256149624>
23. V. Pontillo, D. Amoroso d’Aragona, F. Pecorelli, D. Di Nucci, F. Ferrucci, and F. Palomba, “Machine learning-based test smell detection,” *Empirical Softw. Eng.*, vol. 29, no. 2, p. 55, 2024.
24. J. Withunsapphasiri, P. Phannachitta, and C. Doungsa-Ard, “Predicting test smell categories based on issue descriptions using information retrieval and machine learning techniques,” in *Proc. 2025 Joint Int. Conf. on Digital Arts, Media and Technology with ECTI Northern Section Conf. on Electrical, Electronics, Computer and Telecommunications Engineering (ECTI DAMT & NCON)*, 2025, pp. 125–130, doi: 10.1109/ECTIDAMTNCONE64748.2025.10961982.
25. A. L. Alem, K. K. Gebretsadik, S. A. Mengistie, and M. F. Admas, “Multi-label software requirement smells classification using deep learning,” *Sci. Rep.*, vol. 15, no. 1, p. 5761, Feb. 2025, doi: 10.1038/s41598-025-86673-w.
26. J. Withunsapphasiri, P. Phannachitta, and C. Doungsa-Ard, “Predicting test smell categories based on issue descriptions using information retrieval and machine learning techniques,” in *Proc. 30th Asia-Pacific Software Engineering Conf. (APSEC 2023) — Student Research Competition*, Seoul, South Korea, Dec. 2023. [Poster]. [Online]. Available: <https://conf.researchr.org/track/apsec-2023/apsec-2023-student-research-competition>
27. G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley, “Are test smells really harmful? An empirical study,” *Empirical Softw. Eng.*, vol. 20, no. 4, pp. 1052–1094, 2015, doi: 10.1007/s10664-014-9313-0.

28. W. C. Ouédraogo, Y. Li, K. Kaboré, X. Tang, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé, “Test smells in LLM-generated unit tests,” in *Proc. Int. Conf. on Software Engineering (ICSE)*, 2024, pp. 1–12. [Publisher: IEEE].
29. K. Lucas (Federal University of Campina Grande), M. Ribeiro (Federal University of Campina Grande), R. Gheyi (Federal University of Alagoas), and I. Machado (Federal University of Bahia), “Evaluating large language models in detecting test smells,” *Empirical Softw. Eng.*, pp. 1–20, 2025. [Publisher: Springer].
30. K. Lucas dos Santos, R. Gheyi, M. Ribeiro, and F. Palomba, “Investigating the performance of small language models in detecting test smells in manual test cases,” *J. Syst. Softw.*, pp. 1–15, 2025. [Publisher: Elsevier].
31. S. R. Chidamber and C. F. Kemerer, “A metrics suite for object-oriented design,” *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.
32. S. R. Chidamber and C. F. Kemerer, “Towards a metrics suite for object oriented design,” in *Proceedings of OOPSLA*, pp. 197–211, 1991.
33. D. Bruckner, “RFC: Response set metric for object-oriented systems,” *Journal of OO Development*, 1988.
34. M. Halstead, *Software Metrics*. Amsterdam, Netherlands: Elsevier, 1982.
35. H. N. Khoa, D. T. Binh, and N. T. Binh, “Enhancing test smell prediction with stacking ensemble learning,” in *Proc. 14th Int. Conf. on Information Technology and its Applications (CITA 2025)*, Da Nang, Vietnam, 2025, pp. 1–10.
36. S. R. Chidamber and C. F. Kemerer, “A metrics suite for object-oriented design,” *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.